FitPick

Digital Wardrobe & Outfit Planner

Testing, Version Management and System Build Report

Software Engineering · Final Project Submission (Part A) · Revised

Author **Nithilan A (nithi0307)**
Repository **github.com/nithi0307/fitpick**
Stack **React 19 + Vite · FastAPI · PostgreSQL 16 · Docker**

Revision note

This document has been revised and re-uploaded at the request of the course instructor. After the original submission the application was extended — per-account wardrobes, a ready-made clothing catalogue, removal of items, and saved outfits with a style score — and this report has been updated throughout so that it describes the software as it now stands. Every figure, screenshot and test result below was re-captured from the current build.

Contents

1. Introduction and what changed in this revision
2. Testing strategy and test inventory
3. Integration testing
4. Regression testing
5. Mutation testing
6. Version management
7. System building
8. Screenshots of developed functionality
9. Tools and technologies used

1. Introduction and what changed in this revision

FitPick is a web application for organising a wardrobe and planning outfits. A user signs in, builds a wardrobe either by choosing from a ready-made catalogue or by entering garments by hand, browses and filters it, combines pieces into outfits that are saved and scored, and sees statistics about what they own. It is a **React 19 + Vite** single-page frontend, a **FastAPI** REST backend and a **PostgreSQL 16** database, started together with **Docker Compose**.

1.1 What changed since the first submission

The instructor asked for changes to the application, and this report has been updated to describe what now exists. The functional changes were:

Change	What it means for testing
Per-account wardrobes — every item and outfit belongs to a user row, and signing in with a new email creates an empty wardrobe	Nine new integration tests prove one account cannot read, change or delete another's items
A ready-made catalogue of 33 garments; choosing one copies it into the signed-in wardrobe	Nine new integration tests, including that browsing the catalogue never alters a wardrobe
Items can be removed from the wardrobe, with an in-place confirmation	Covered by the existing delete tests plus the per-account deletion tests
Saved outfits appear as Fit 1, Fit 2 … each with a style score out of ten	A new rule-based service with 32 unit tests, and a second mutation-testing target
An account menu, a home shortcut on the logo, and 46 clothing subcategories	Interface-level; verified by hand and captured in Section 8

The effect on the test suite is the headline number: it has grown from **46 tests to 96**, and mutation testing now covers two service modules instead of one.

1.2 Scope of this document

This is the evidence document for the build and quality side of the project. It covers:

1. the automated tests — integration, regression and mutation testing — with the terminal output of each run;
2. version management and system building — the Git workflow, repository state and the containerised build;
3. screenshots of every screen of the running application;
4. the tools and technologies used, and the ones deliberately left out.

Every screenshot was captured from the current build on the machine the project was developed on. The design itself — architecture, data flow, ERD, UML — is in the companion **FitPick Project Report**.

1.3 What was built

Area	Delivered
Frontend	7 screens (Login, Dashboard, My Wardrobe, Add Clothing, Outfit Builder, My Fits, Profile & Analytics), 11 reusable components, 1 API client, 1 session module, 1 design-system stylesheet
Backend	4 routers (clothing, outfits, catalogue, stats), 17 REST endpoints, Pydantic schema layer, 2 services (analytics, style scoring), repository layer with in-memory and PostgreSQL implementations
Database	4 tables (users, clothing_items, outfits, outfit_items) with schema and seed scripts
Catalogue	33 predefined garments across six categories, 20 SVG garment illustrations
Tests	96 automated tests across three levels, plus mutation testing over both services
Build	3 containers for development, 1 production image serving the API and the compiled app

2. Testing strategy and test inventory

Testing concentrates on the backend, because that is where the application's rules live: identity, filtering, validation, storage, the dashboard figures and the outfit score. The frontend is checked by **ESLint** and a production build on every change, and verified by hand against the running API.

2.1 Test levels used

Level	Purpose	Where
Integration	Drive the real FastAPI application over HTTP so routing, identity, validation, serialisation and storage are exercised together.	tests/test_integration_*.py
Regression	Lock in behaviour for edge cases that were wrong, or easy to get wrong, so the same fault cannot return.	tests/test_regression.py
Unit	Check the two service modules directly, without HTTP. Most of these were written in response to mutation testing.	tests/test_analytics_unit.py tests/test_style_unit.py
Mutation	Measure how good the tests actually are by deliberately introducing faults and checking that a test fails.	mutmut, configured in setup.cfg

2.2 Test inventory

Test file	Level	Tests	What it covers
test_integration_clothing.py	Integration	12	Create, read, update, delete, search, filter, sort, favourite, validation
test_integration_outfits.py	Integration	7	Listing, creating, rejecting empty or unknown items, delete, favourite
test_integration_stats.py	Integration	6	Dashboard totals, most-worn item, breakdown consistency, wardrobe value
test_integration_accounts.py	Integration	9	Per-account wardrobes and isolation between accounts
test_integration_catalogue.py	Integration	9	Browsing the catalogue and copying a piece into a wardrobe
test_regression.py	Regression	10	Ten specific defects and edge cases (Section 4)
test_analytics_unit.py	Unit	11	Counting, ordering, date boundaries, rounding, empty wardrobe
test_style_unit.py	Unit	32	Every rule of the outfit style score
Total		96	

2.3 How the suite is run

From the backend/ directory

```
$ python3.13 -m venv .venv
$ .venv/bin/pip install -r requirements-dev.txt
$ .venv/bin/python -m pytest               # all 96 tests
$ .venv/bin/mutmut run                     # mutation testing over services/
```

Tests never touch a database. The repository factory in `repositories/__init__.py` returns the in-memory implementation unless `DATABASE_URL` is set, so the whole suite runs in well under a second and gives the same result on any machine.

backend/conftest.py — every test starts from a freshly seeded wardrobe

```
import pytest
from fastapi.testclient import TestClient

import repositories
from main import app


@pytest.fixture
def client() -> TestClient:
    repositories.reset()
    with TestClient(app) as test_client:
        yield test_client
    repositories.reset()
```

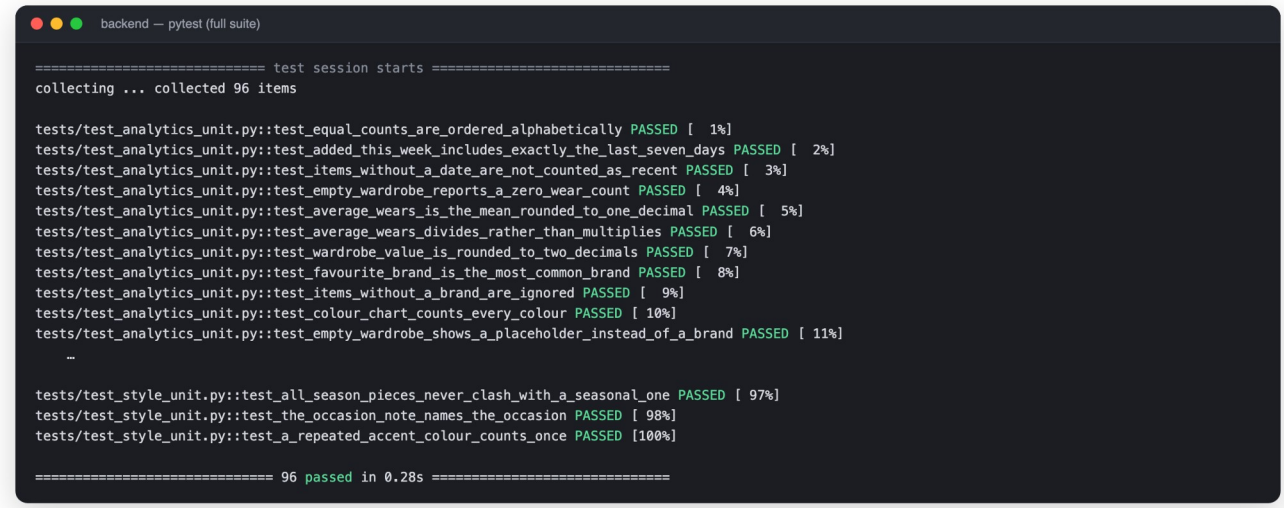


Figure 1 — The complete suite: 96 tests passing.

3. Integration testing

The integration tests use FastAPI's `TestClient`, which sends real HTTP requests into the real application object. Nothing is mocked: a request passes through identity resolution, the router, Pydantic validation, the repository and back out as JSON, exactly as it does for the React frontend. That is what makes them **integration** rather than unit tests — several modules are proved to work together.

3.1 What each group covers

- **Clothing (12) —** the full CRUD lifecycle, plus the query parameters the My Wardrobe screen sends: search text, category, season and sort order. Two tests confirm invalid input is rejected with HTTP 422 rather than stored.
- **Outfits (7) —** creating an outfit from existing wardrobe items, and the two rules that protect the data: an outfit must contain at least one item, and every item id must exist.
- **Statistics (6) —** the figures on the Dashboard are cross-checked against the wardrobe itself, so the totals cannot drift from the underlying data.
- **Accounts (9) —** a new email starts with an empty wardrobe, and one account can neither read nor delete another account's items.
- **Catalogue (9) —** browsing never changes a wardrobe, and choosing a piece copies it into the caller's wardrobe only.

backend/tests/test_integration_clothing.py — a representative test

```
def test_add_item_then_read_it_back(client):
    created = client.post("/clothing", json=NEW_ITEM)
    assert created.status_code == 201
    item_id = created.json()["id"]

    fetched = client.get(f"/clothing/{item_id}")
    assert fetched.status_code == 200
    assert fetched.json()["name"] == "Linen Shorts"
    assert fetched.json()["brand"] == "Arket"
```

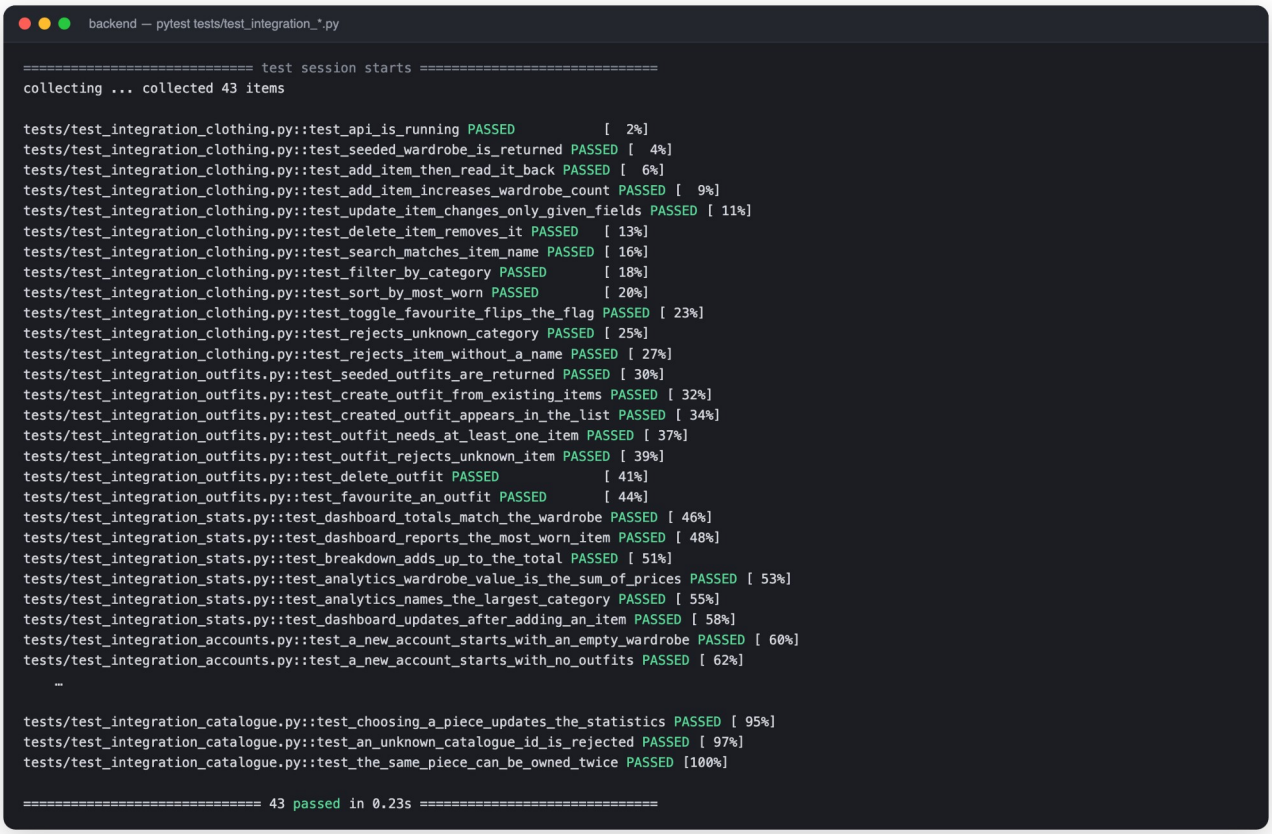


Figure 2 — The 43 integration tests passing. Each name states the behaviour it proves.

3.2 Testing the two new features

Per-account wardrobes and the catalogue are the two changes with the greatest risk of leaking data between users, so both were tested at the boundary that matters — the HTTP interface, using the `X-User-Email` header the frontend sends:

backend/tests/test_integration_accounts.py — isolation between accounts

```
def test_an_item_added_by_one_account_belongs_only_to_it(client):
    created = client.post("/clothing", json=ITEM, headers=NEW)
    assert created.status_code == 201

    assert [i["name"] for i in client.get("/clothing", headers=NEW).json()] == ["Linen Shirt"]
    assert client.get("/clothing", headers=OTHER).json() == []
    assert len(client.get("/clothing").json()) == 12


def test_one_account_cannot_read_another_accounts_item(client):
    item_id = client.post("/clothing", json=ITEM, headers=NEW).json()["id"]
    assert client.get(f"/clothing/{item_id}", headers=OTHER).status_code == 404
```

backend/tests/test_integration_catalogue.py — browsing must be read-only

```
def test_browsing_the_catalogue_does_not_change_the_wardrobe(client):
    client.get("/catalogue", headers=NEW)
    assert client.get("/clothing", headers=NEW).json() == []
```

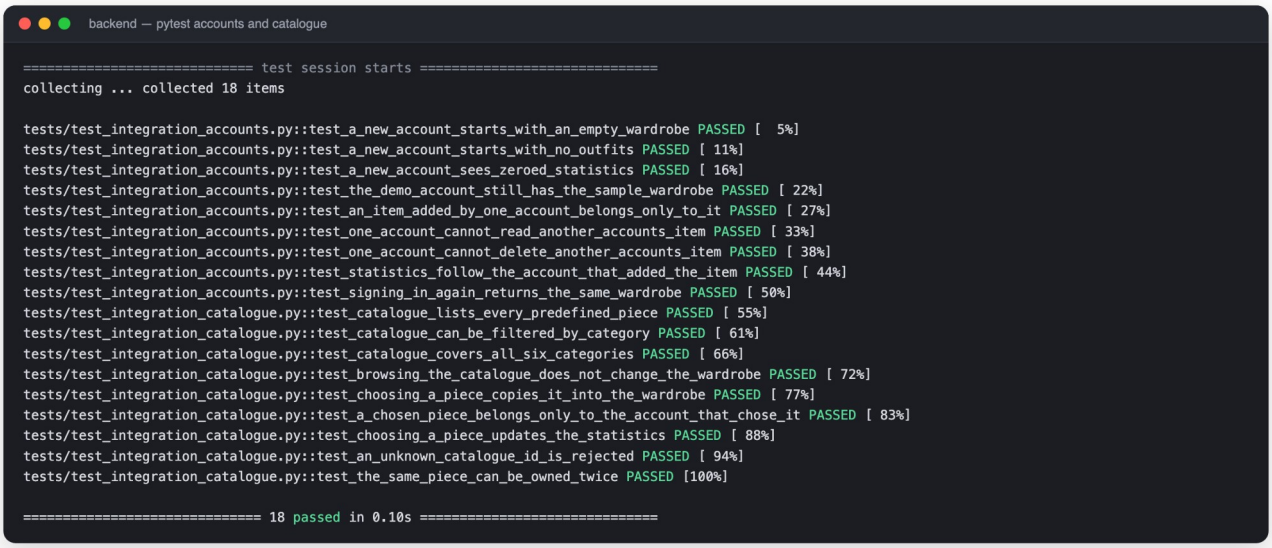


Figure 3 — The 18 tests covering accounts and the catalogue.

4. Regression testing

A regression test exists to stop a fault coming back. Each of the ten tests below was written after a specific edge case was found during development; they run on every change, so if the behaviour is ever broken again the suite fails immediately.

ID	Behaviour locked in	Why it was needed
REG-01	Search matches the brand as well as the item name	Search originally looked at the name only, so searching “Levi” returned nothing
REG-02	Deleting an unknown id returns HTTP 404	A missing item must not be reported as a successful delete
REG-03	Category “All” is treated as “no filter”	The literal string was once matched against the category column
REG-04	Analytics survives an empty wardrobe	Average wears divided by the item count and crashed on a new account
REG-05	Dashboard survives an empty wardrobe	max() over an empty list raises ValueError
REG-06	Favouriting twice restores the original state	The toggle must be symmetric
REG-07	A partial update keeps the fields it was not given	An update once blanked every omitted field
REG-08	Search is case-insensitive	“NAVY” and “navy” must return the same items
REG-09	Counts are ordered largest-first	The breakdown chart depends on descending order
REG-10	Deleting an item updates the statistics	Statistics must never be served from a stale copy

REG-04 and REG-05 became more important after this revision, not less: with per-account wardrobes, **every new user now begins with exactly the empty wardrobe these two tests describe**. What was once an edge case is now the first thing every user sees.

backend/tests/test_regression.py — the empty-wardrobe cases

```
def test_reg04_analytics_handles_an_empty_wardrobe(empty_wardrobe):
    """Average wears divided by the item count and crashed on a new account."""
    clothing, _ = empty_wardrobe
    stats = analytics_service.analytics_stats(clothing)
    assert stats.average_wears == 0.0
    assert stats.wardrobe_value == 0.0
    assert stats.top_category == "—"


def test_reg05_dashboard_handles_an_empty_wardrobe(empty_wardrobe):
    """max() over an empty wardrobe raised ValueError on the dashboard."""
    clothing, outfits = empty_wardrobe
    stats = analytics_service.dashboard_stats(clothing, outfits)
    assert stats.total_items == 0
    assert stats.most_worn == "—"
    assert stats.breakdown == []
```

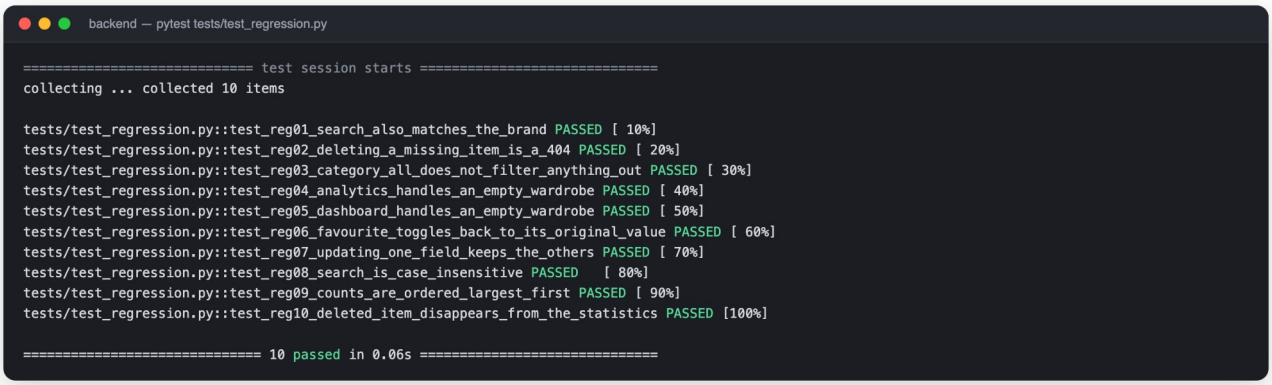


Figure 4 — All ten regression tests passing.

5. Mutation testing

A suite that passes proves the code runs; it does not prove the tests would notice if the code were wrong. **Mutation testing** answers that directly. The tool makes small deliberate changes to the source — a >= becomes >, a - becomes + — and runs the suite against each one. If a test fails, the mutant is **killed**. If every test still passes, the mutant **survived**, meaning that behaviour is not really being checked.

5.1 Setup

The tool is `mutmut`. It is pointed at the whole `services/` package, which is now two modules: `analytics_service.py`, which produces every figure on the Dashboard and Analytics screens, and `style_service.py`, which scores a saved outfit out of ten. Both are pure logic with no I/O, so a fault there would be visible to the user but would not crash anything — exactly the kind of fault a test suite can miss.

backend/setup.cfg

```
[mutmut]
paths_to_mutate=services/
runner=.venv/bin/python -m pytest -x -q --no-header
tests_dir=tests/
```

5.2 First run on the revised code

Running the tool over both services generated **131 mutants**. Every mutant in the analytics service was killed — those tests had already been hardened in the first submission — but the newly written style service left **27 alive**:

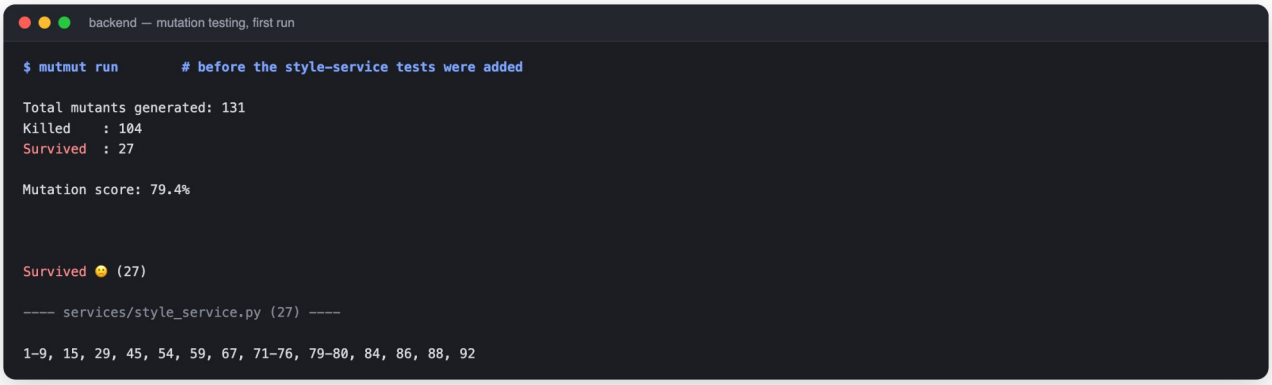


Figure 5 — First run after the revision: 104 of 131 killed, a mutation score of 79.4%.

5.3 What the survivors exposed

Each survivor was inspected with mutmut show, and each one pointed at a real gap in the new tests:

Mutants	Change the tool made	Gap it exposed
1–9	Each neutral colour removed from the NEUTRALS set	No test proved that every listed neutral is actually treated as neutral
29	points += 1 → points = 1 in the coverage rule	Nothing checked that coverage points accumulate rather than overwrite
45	“Accessories” altered in the coverage rule	No outfit in the tests was completed by an accessory rather than outerwear
54, 59, 67	The wording of the “Missing …” note	The explanation shown to the user was never asserted
71–76, 79–80	The accent-colour thresholds and their scores	Only the three-accent case was tested; one and two accents were not
84, 86, 88	The “All Season” literal and the season note	No outfit mixed an All-Season piece with a seasonal one
92	The wording of the occasion note	The positive occasion case was never asserted

5.4 Closing the gaps

Twenty unit tests were added to `tests/test_style_unit.py`, each written to kill a specific survivor. They are genuine tests, not padding — the parametrised one below, for example, proves every colour the service calls neutral really is treated as one:

backend/tests/test_style_unit.py — kills mutants 1 to 9

```
@pytest.mark.parametrize(
    "colour", ["White", "Black", "Grey", "Gray", "Navy", "Beige", "Cream", "Brown", "Tan"]
)
def test_every_listed_neutral_counts_as_neutral(colour):
    outfit = [
        piece("Tops", colour, tags=["Casual"]),
        piece("Bottoms", colour),
        piece("Shoes", colour),
    ]
    assert score_outfit(outfit, "Casual")[1][1] == "Neutral palette"
```

backend/tests/test_style_unit.py — kills mutant 29

```
def test_coverage_points_accumulate_rather_than_overwrite():
    shoes_only = [piece("Shoes", "Black", tags=["Casual"])]
    full_base = [
        piece("Tops", "Black", tags=["Casual"]),
        piece("Bottoms", "Black"),
        piece("Shoes", "Black"),
    ]
    assert score_outfit(shoes_only, "Casual")[0] == 1 + MAX_COLOUR + MAX_SEASON + MAX_OCCASION
    assert score_outfit(full_base, "Casual")[0] == 3 + MAX_COLOUR + MAX_SEASON + MAX_OCCASION
```

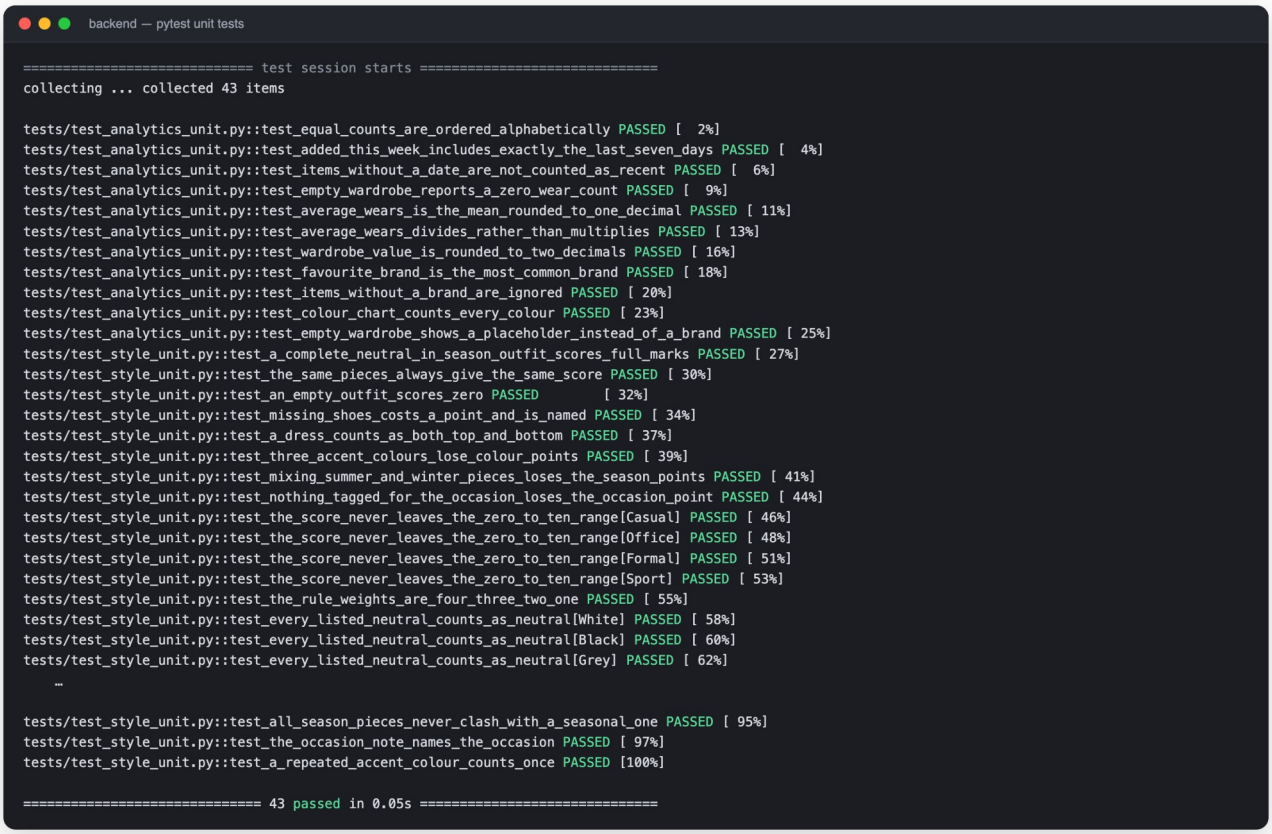


Figure 6 — The 43 unit tests over the two services.

5.5 Second run



Figure 7 — Second run: 129 of 131 killed, a mutation score of 98.5%.

5.6 The two remaining survivors

Neither is a gap in the tests, and both were checked by hand rather than assumed:

Mutant	Change	Finding
#15	The four weighting constants replaced with None	The suite does fail on this — the module raises TypeError on import. mutmut 2.5 classifies a collection error as “survived” rather than “killed”, so this is a limitation of the tool, not of the tests. Verified by applying the change by hand and running pytest.
#29	points += 1 → points = 1, on the first branch only	An equivalent mutant: points was set to 0 on the line above, so the two statements have identical behaviour. No test can distinguish them, and none should.

Outcome

Mutation score over both services rose from 79.4% (104/131) to 98.5% (129/131).
Of the two survivors, one is caught by the suite but misreported by the tool, and one is provably equivalent — so every killable mutant is killed.
The suite grew from 76 to 96 tests, and the twenty new tests were chosen by evidence rather than guesswork.
No production code had to change: the scoring logic was already correct, but twenty of its behaviours were unverified.

6. Version management

The project is version-controlled with **Git** and hosted on **GitHub** at `github.com/nithi0307/fitpick`. Work is tracked as GitHub issues prioritised with MoSCoW on a project board.

6.1 Branching strategy

Branch	Role
`main`	Always runnable. Every commit on it builds and passes the test suite.
`feature-<name>`	One branch per feature or issue, merged back into main when complete.

6.2 Commit history

Commits are small and each has a single purpose, so the history reads as a description of how the project was built — including the revision work described in Section 1:

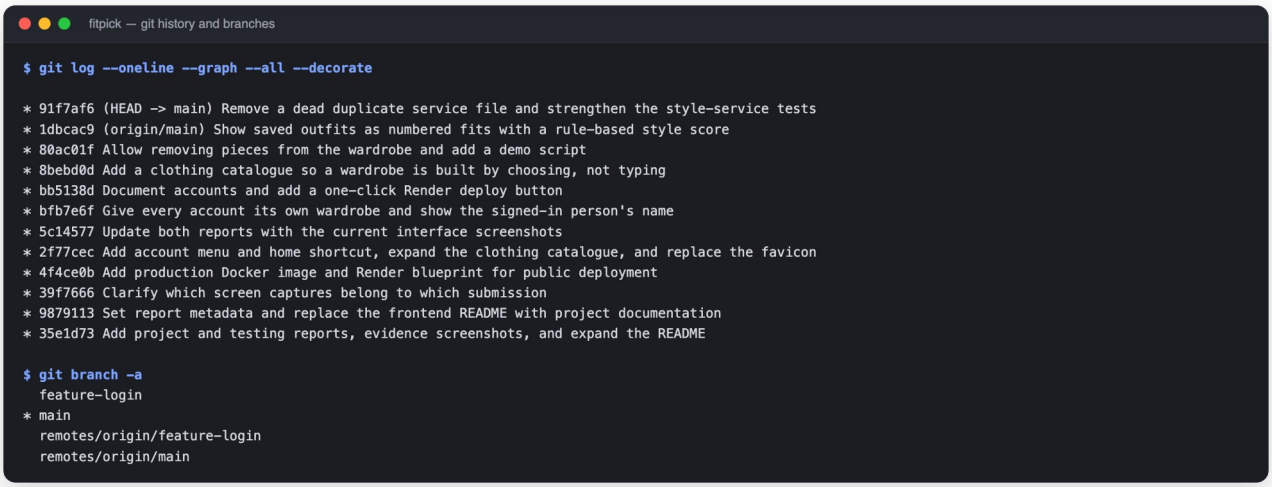


Figure 8 — Commit graph and branches.

6.3 Repository hygiene

Two clean-ups came out of this revision. Early in the project a dependency folder and macOS metadata files had been committed by accident; they were removed and a `.gitignore` now keeps the repository to source files only. Second, mutation testing surfaced a file that should never have been there:

A dead file found by the tooling

The first mutation run over services/ reported 30 survivors in a file called “analytics_service 2.py” — a macOS duplicate that had been committed by accident. Nothing referenced it, and the space in its name means Python cannot even import it. It was dead weight that made the mutation score look 15 points worse than it was. It has been deleted, and the score in Section 5 is measured on live code only.

fitpick/.gitignore

```
# Dependencies
node_modules/

# Build output
dist/
build/

# Python
__pycache__/
*.pyc
.venv/
venv/

# Environment
.env
.env.*

# OS / editor
.DS_Store
.vscode/
.idea/

# Logs
*.log
.mutmut-cache
mutants/
.pytest_cache/
```

6.4 The repository on GitHub

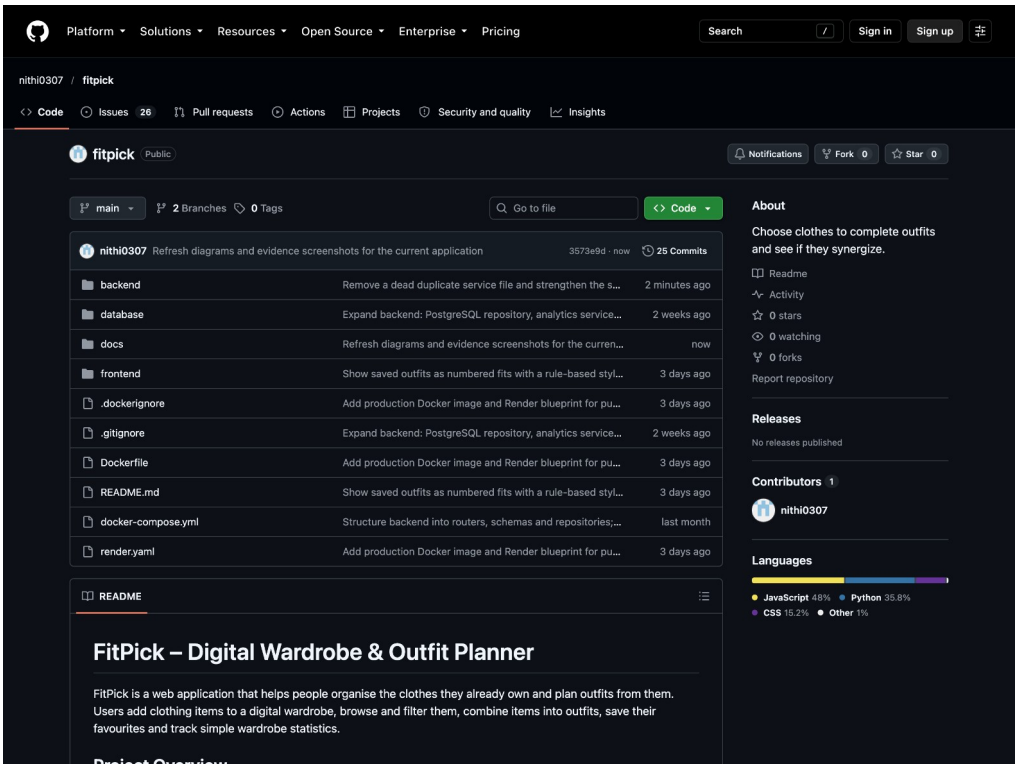


Figure 9 — The repository: a single contributor and the README rendered on the landing page.

7. System building

There are two builds. **Development** uses `docker-compose.yml` — three containers with hot reload. **Production** uses a multi-stage `Dockerfile` that compiles the React app and serves it from the FastAPI container, so the deployed system is a single service with the site and its API on one origin and no CORS configuration at all.

fitpick/Dockerfile — the production image

```
# Production image: builds the React app, then serves it from the FastAPI
# container so the whole system is one deployable unit with no CORS coupling.
# Local development still uses docker-compose.yml (two containers, hot reload).

# ---------- stage 1: build the frontend ----------
FROM node:20-alpine AS web

WORKDIR /web
COPY frontend/package*.json ./
RUN npm ci

COPY frontend/ ./
RUN npm run build

# ---------- stage 2: the application ----------
FROM python:3.13-slim

WORKDIR /app

COPY backend/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ ./
COPY database/ ./database/
COPY --from=web /web/dist ./static

EXPOSE 8000

# Render, Railway and Fly all supply $PORT; default to 8000 elsewhere.
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

7.1 Running the development stack

The database port is published as 5433 on the host because a local PostgreSQL already occupies 5432; inside the container it stays on the standard 5432, so no application configuration changes.

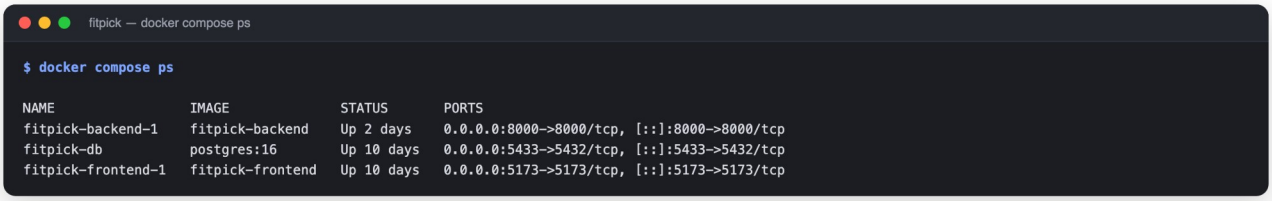


Figure 10 — The three containers running.

7.2 Per-account data in the database

The schema always carried a `user_id` foreign key; this revision made the application use it. Each account now owns its own rows, and a first sign-in creates the user record automatically — in the deployed image the application also applies `schema.sql` and seeds an empty database on first start, both idempotently:



Figure 11 — Accounts and their item counts, queried inside the running container.

7.3 The API in the built environment

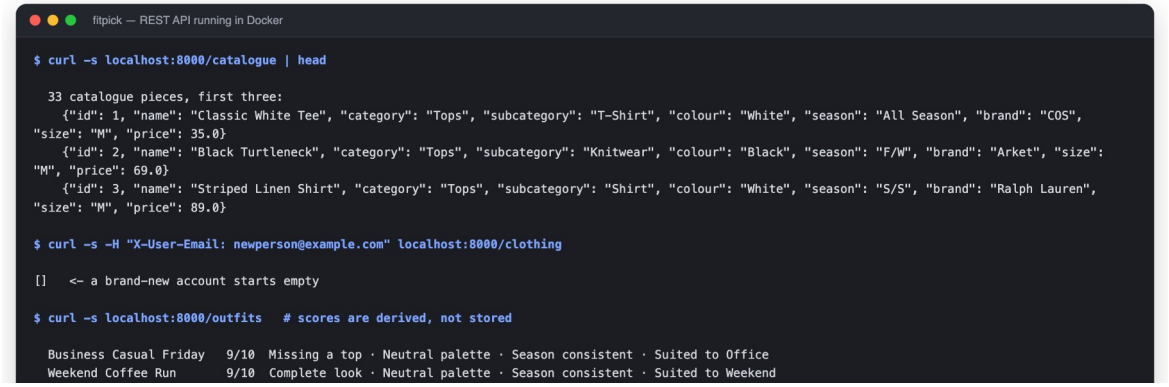


Figure 12 — Smoke test: the catalogue, an empty wardrobe for a new account, and outfit scores derived on read.

7.4 Generated API documentation

FastAPI generates an OpenAPI description from the Pydantic schemas, so the API documents itself at `/docs`. This is the contract the React frontend is written against:

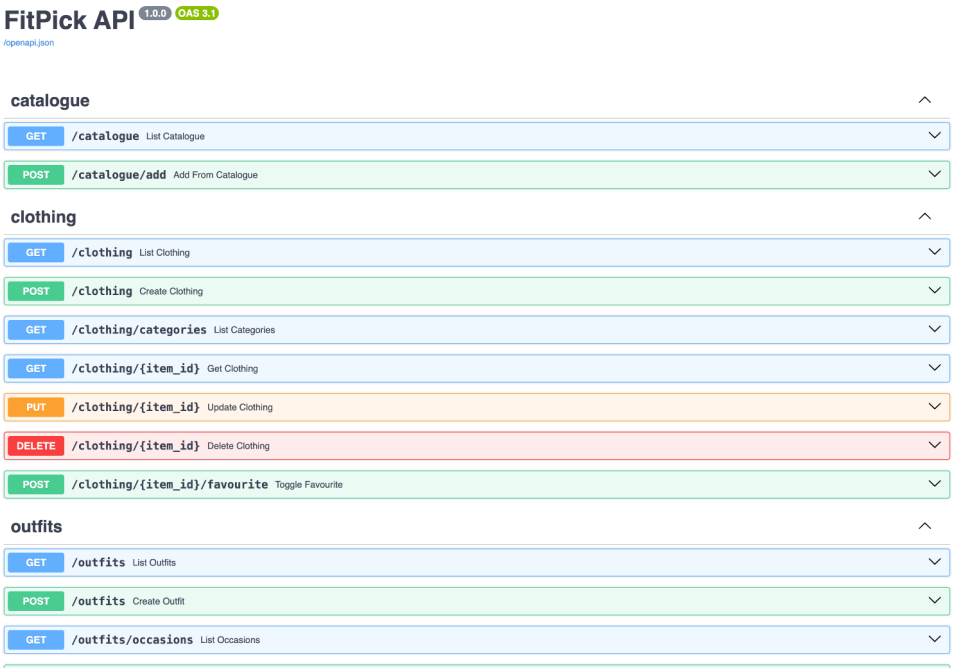


Figure 13 — The REST endpoints, generated from the code.

7.5 Build instructions

Development — three containers with hot reload

```
$ git clone https://github.com/nithi0307/fitpick.git
$ cd fitpick
$ docker compose up --build

#  frontend     http://localhost:5173
#  backend API  http://localhost:8000   (docs at /docs)
#  PostgreSQL   localhost:5433  db fitpick  user postgres
```

Production — one image, site and API on the same origin

```
$ docker build -t fitpick .
$ docker run -p 8000:8000 -e DATABASE_URL=postgresql://... fitpick

#  render.yaml provisions the same image plus a PostgreSQL instance
```

8. Screenshots of developed functionality

All seven screens are implemented in React and were captured from the running application with the backend and PostgreSQL serving real data. The design system — royal blue #3B6FF6 on white cards over a light grey ground — is shared by every screen.

8.1 Login / Sign Up

A split screen: the form on the left, the product promise on the right. One form serves both sign-in and sign-up. The name and email entered here become the signed-in identity, which the sidebar, avatar and greeting all read.

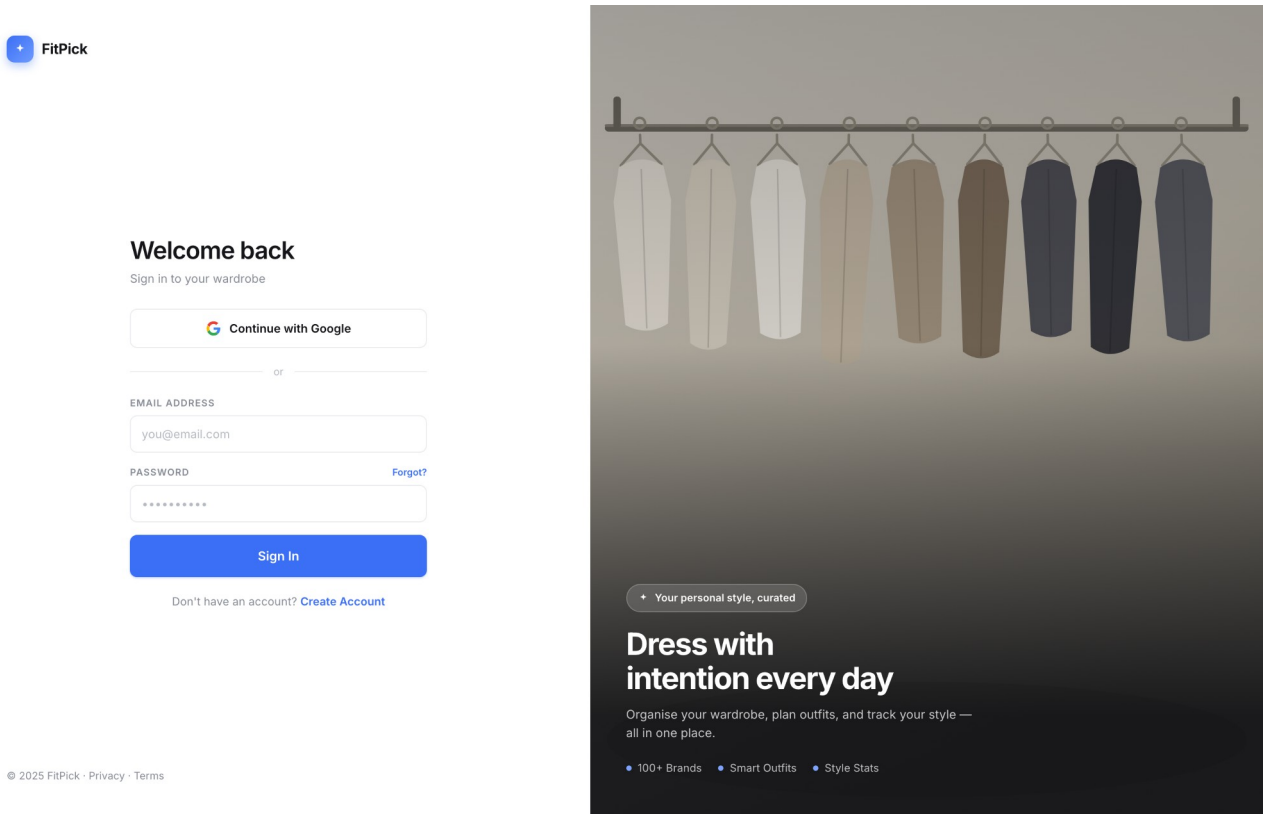


Figure 14 — Login / Sign Up

8.2 Dashboard

Four live figures across the top, recent additions and saved fits on the left, quick actions and the wardrobe breakdown on the right. Every number is calculated by the backend from the signed-in account's own wardrobe.

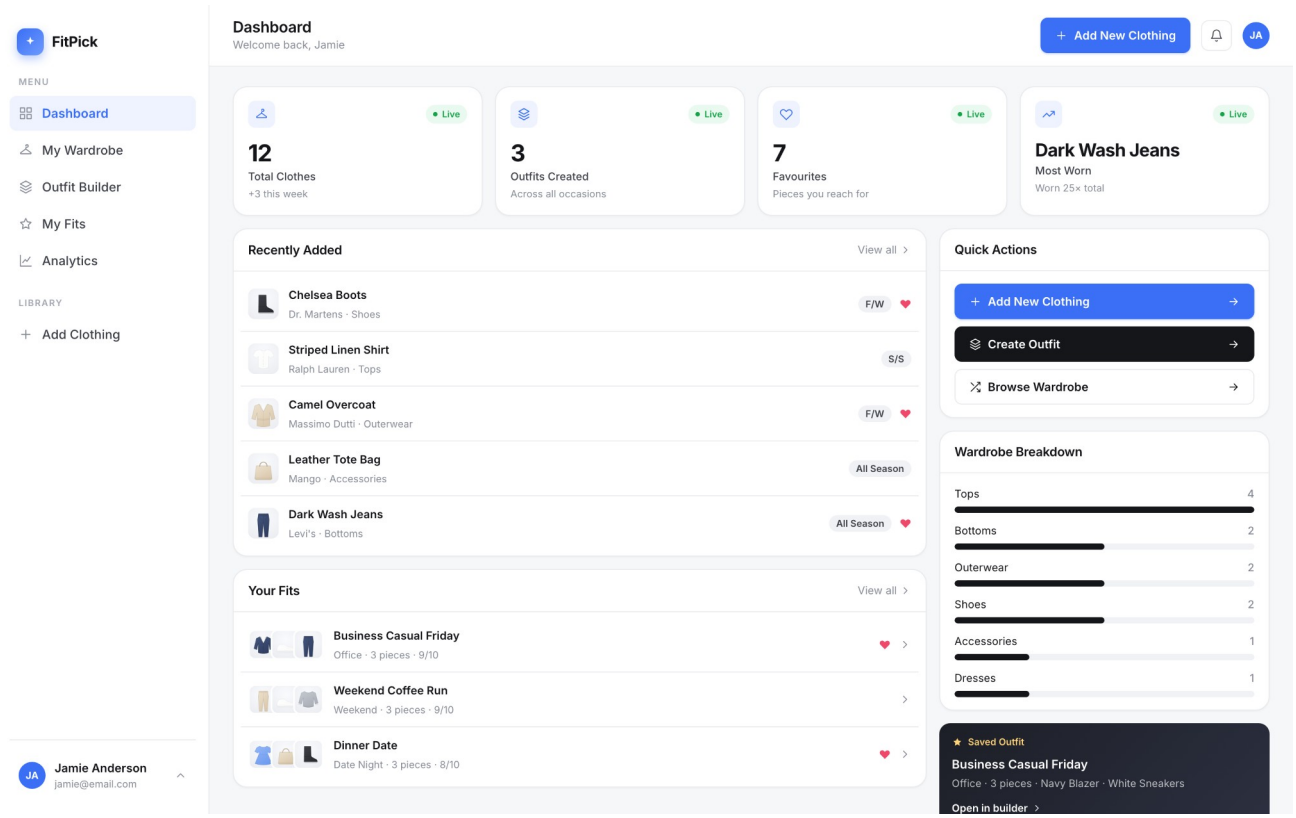


Figure 15 — Dashboard

8.3 The clothing catalogue

Part of the catalogue carried on the Dashboard: 33 ready-made garments across the six categories. Browsing changes nothing; choosing a piece copies it into the wardrobe. Pieces already owned carry a green badge and the button becomes “Add another”.

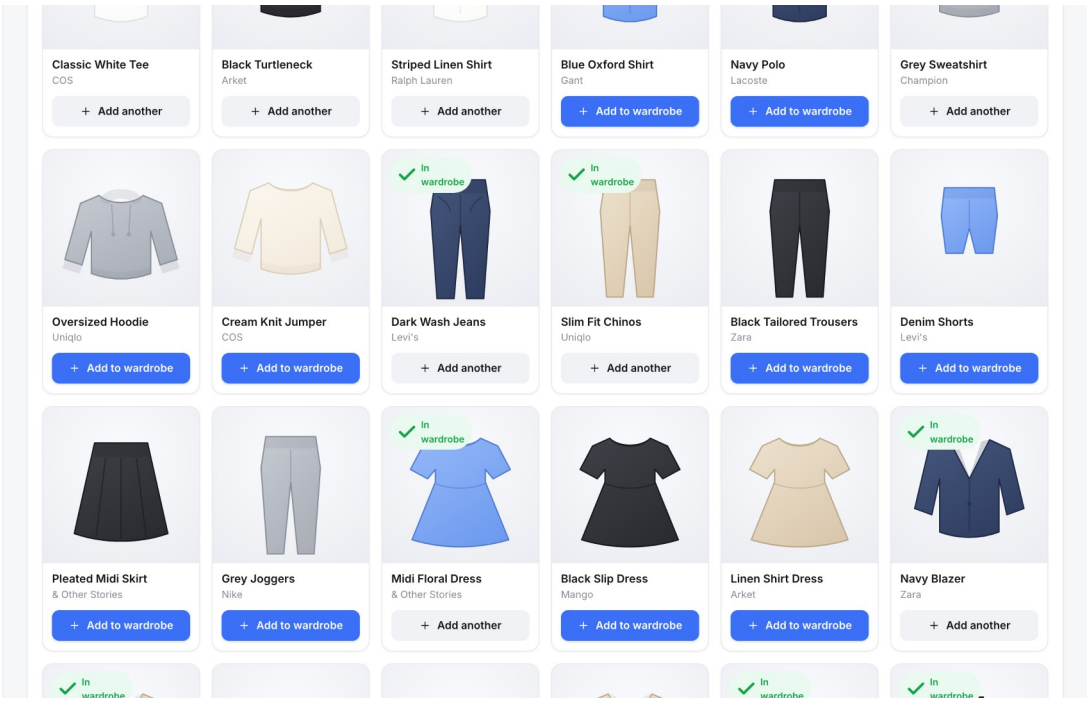


Figure 16 — The clothing catalogue

8.4 My Wardrobe

The collection as a card grid, with search, category and season filters and a sort order. Filtering is done by the API. Hovering a card reveals a remove control, which asks for confirmation in place so one click is never destructive.

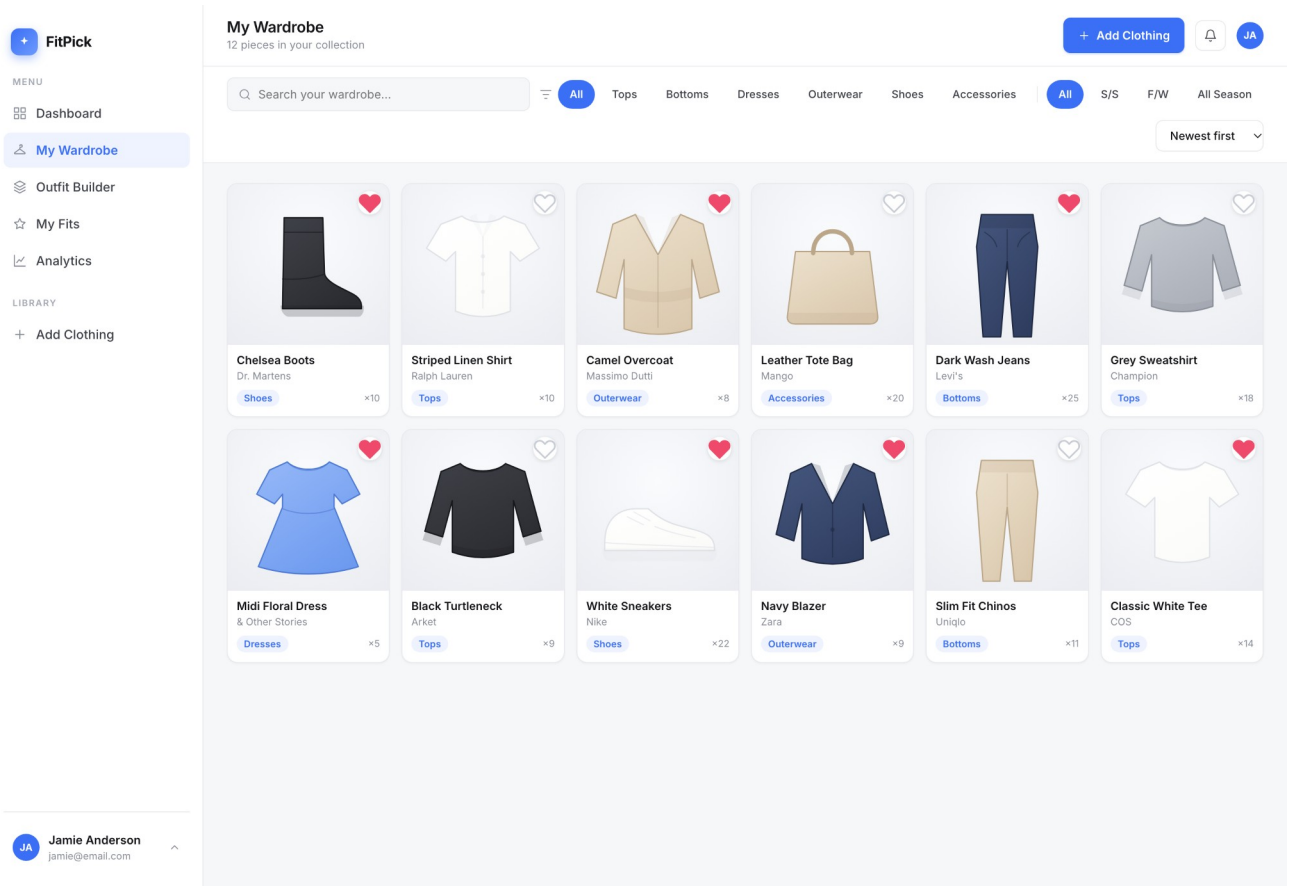


Figure 17 — My Wardrobe

8.5 Add Clothing Item

The manual capture form for anything not in the catalogue: name, category and subcategory, colour, season, brand, size, price paid, tags and notes. The subcategory list is driven by the chosen category and covers 46 garment types.

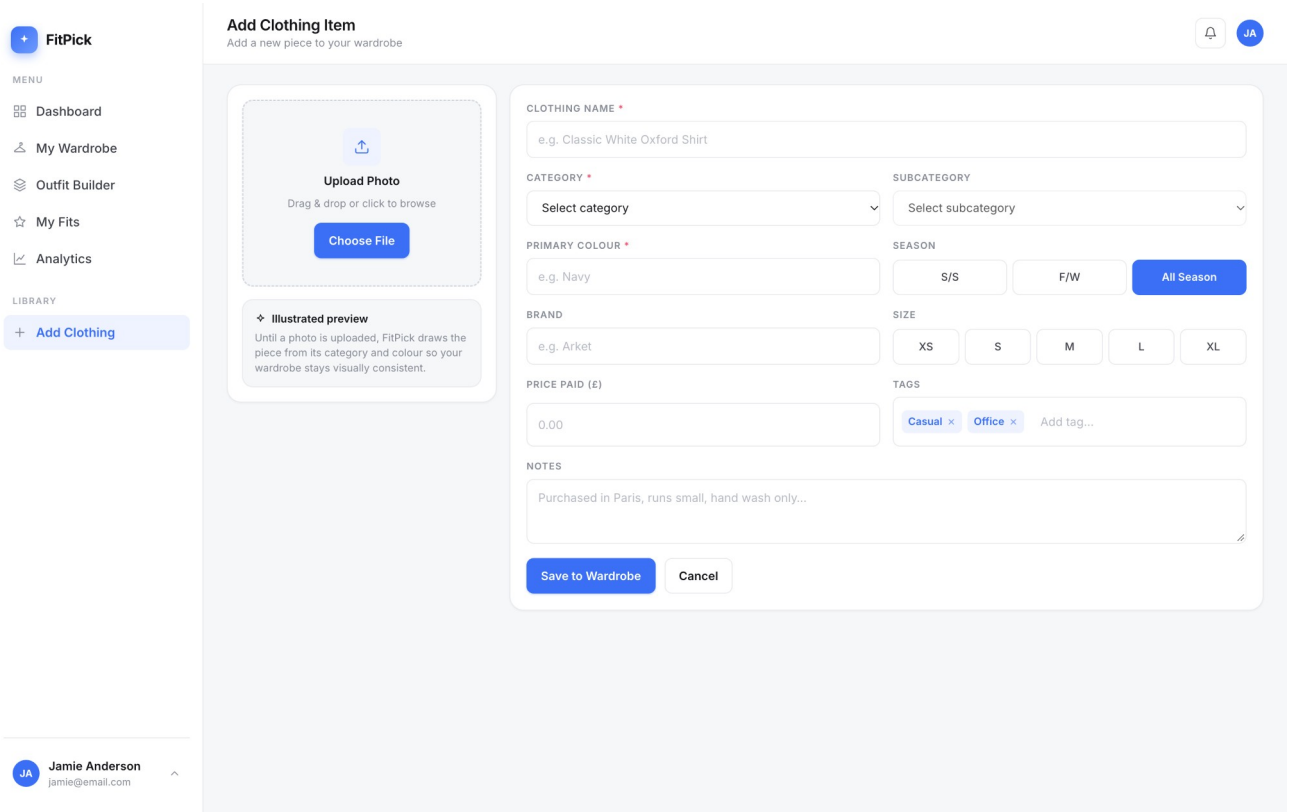


Figure 18 — Add Clothing Item

8.6 Outfit Builder

Three columns — pick pieces on the left, see the outfit take shape in the middle, name it and set its occasion on the right. Opening it with ?outfit=<id> loads a saved outfit for editing.

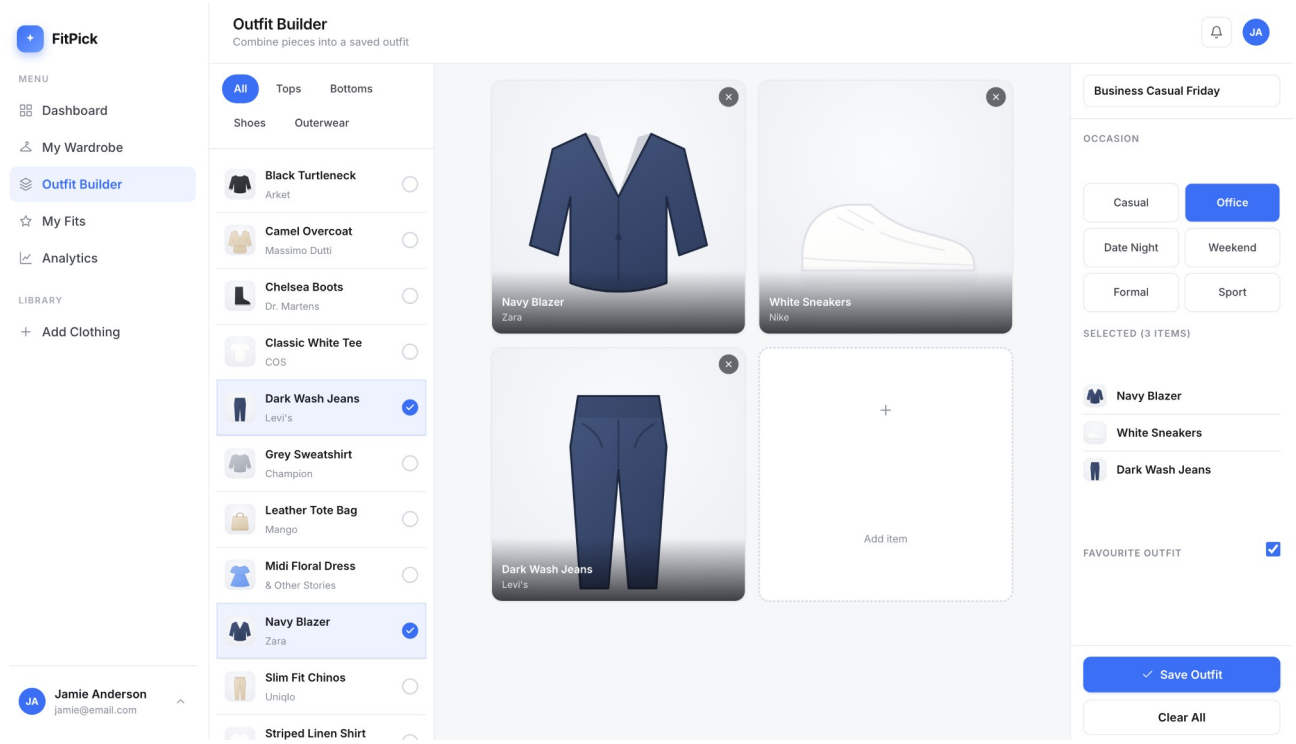


Figure 19 — Outfit Builder

8.7 My Fits

Every saved outfit, numbered Fit 1, Fit 2 … with its style score out of ten and the reason for each part of that score. From here a fit can be favourited, reopened in the builder, or deleted.

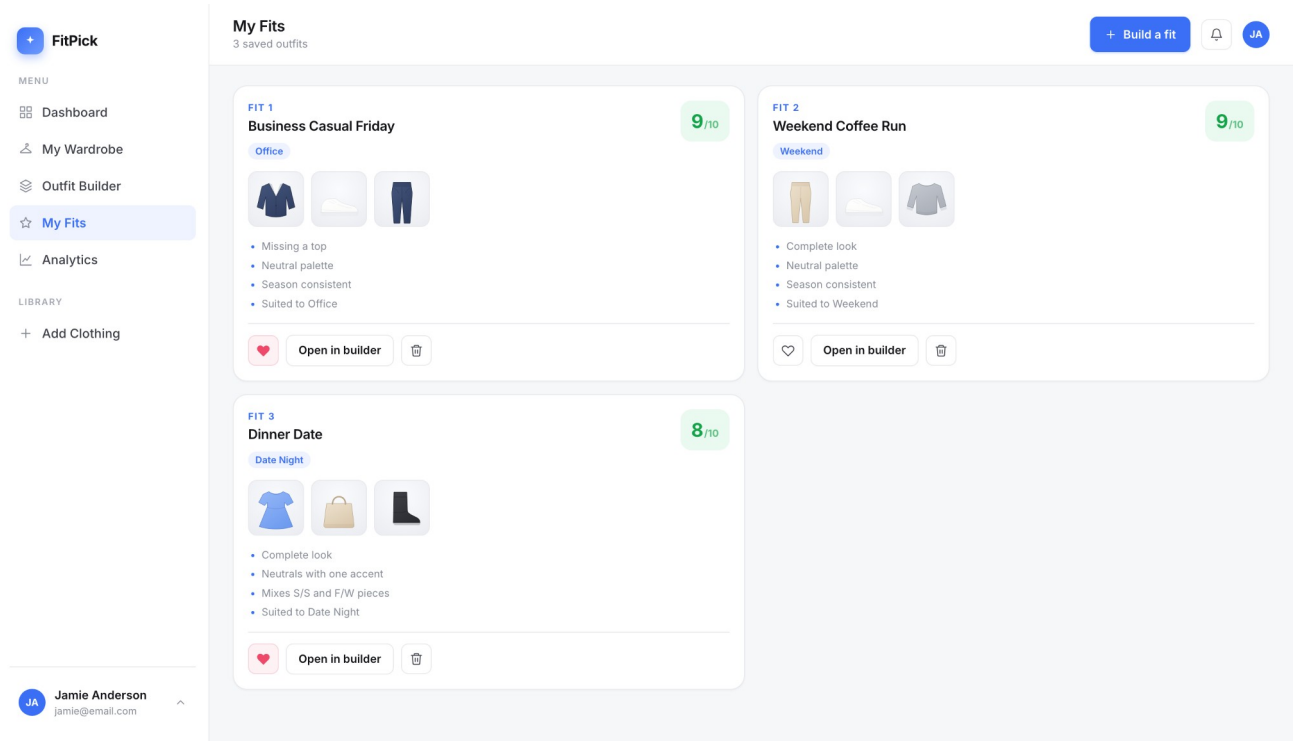


Figure 20 — My Fits

8.8 Profile & Analytics

Account details beside wardrobe insight: total value, average wears per item, a category donut and a colour breakdown, all computed from the signed-in account's own items. The account chip and the top-bar avatar open the same menu, offering profile and log out.

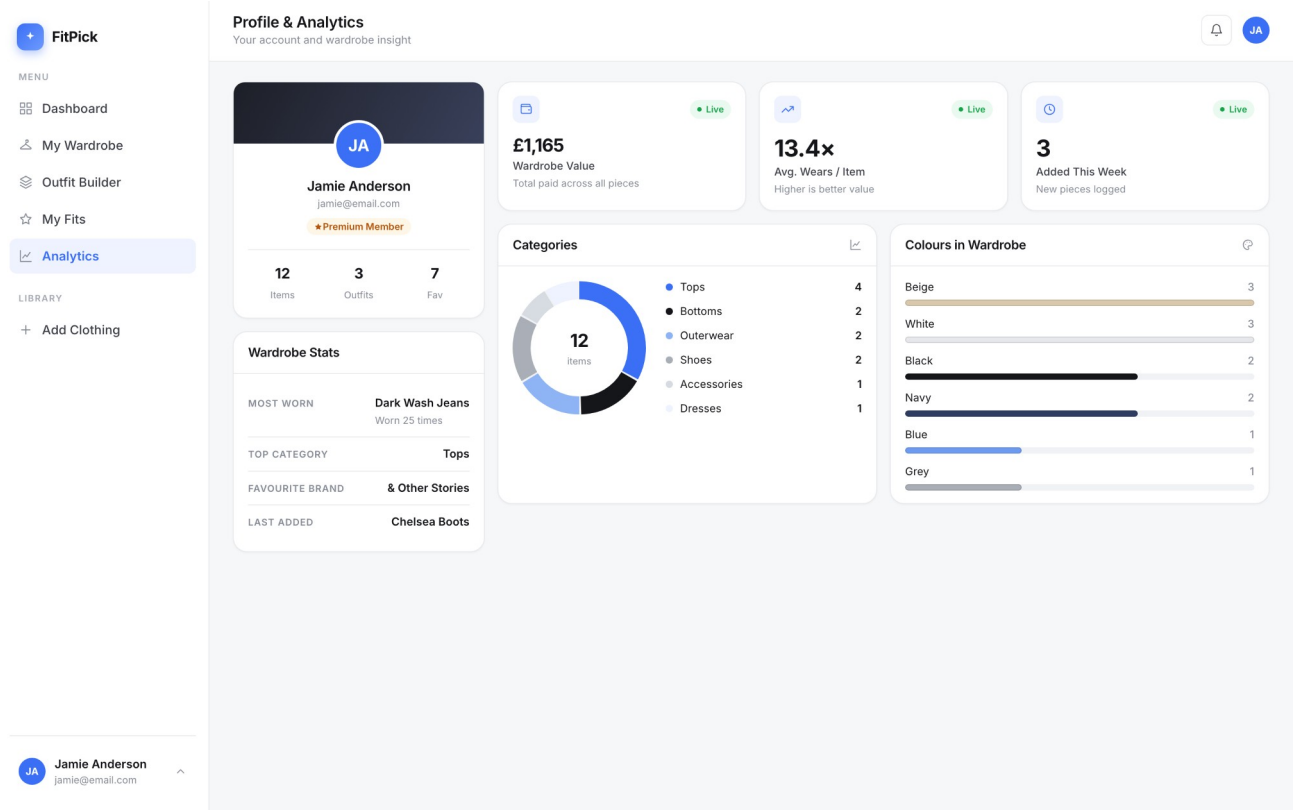


Figure 21 — Profile & Analytics

9. Tools and technologies used

9.1 Application stack

Layer	Technology	Version	Why it was chosen
Frontend	React + Vite	19.2 / 8.2	Component model maps onto the designed screens; Vite gives instant rebuilds
Routing	React Router	7.18	Declarative route table; nested routes give every screen the sidebar automatically
Backend	FastAPI	0.115	Small typed REST framework with validation and OpenAPI documentation built in
Validation	Pydantic	2.x	Models double as the API contract, so invalid data is rejected before reaching storage
Server	Uvicorn	0.34	ASGI server used in development and inside the container
Database	PostgreSQL	16	Relational data: users own items, outfits join items many-to-many
DB driver	psycopg	3.2	Modern PostgreSQL driver with native support for arrays and dict rows
Containers	Docker Compose	v2	One command builds and starts all services identically on any machine

9.2 Quality and engineering tools

Purpose	Tool	How it is used
Test runner	pytest 8.3	Runs all 96 tests; configuration in pytest.ini
HTTP test client	Starlette TestClient / httpx	Sends real requests into the application for the integration tests
Mutation testing	mutmut 2.5	Measures test quality over both modules in services/
Linting	ESLint 10	Checks the frontend on every change (npm run lint)
Build	Vite build	Produces the production bundle and catches import errors
Version control	Git + GitHub	Branching, history, issues and the MoSCoW project board
Design	Figma	The screens were designed before being implemented
Diagrams	Draw.io (diagrams.net)	Architecture, DFD, ERD and UML; editable sources in docs/design/

9.3 Deliberate exclusions

A small stack was chosen on purpose. The following were considered and left out because they would add work and risk without serving any user story in this version:

- No AI or machine learning anywhere. The outfit score is a rule-based model in services/style_service.py — deterministic, explainable and unit-tested. Calling it AI would have been easy and untrue.
- No JWT, OAuth or session store. The email identifies an account so wardrobes stay separate; it does not authenticate one. Hashed passwords and sessions are listed as future work.
- No Nginx or reverse proxy — in production the FastAPI container serves the compiled app itself.
- No micro-services, message queue or cache — a three-tier layered design is the right size for this system.
- No state-management library (Redux, Zustand) — React's own state is enough for seven screens.
- No charting library — two chart shapes did not justify roughly 100 kB of dependency.

Nothing in this report is claimed that is not in the repository. The code, the 96 tests, the diagrams and every screenshot are at `github.com/nithi0307/fitpick`.